

Building a vault from your own manuals

APPLIES TO Field Assist with the manual tier (September 2026 builds)

LICENCE Team licence code required

FORMATS PDF (text layer or scanned), EPUB, Markdown, text

A vault is a folder. It holds a manifest that names things, a few short markdown files the assistant always has in front of it, and the OEM manuals it searches when a technician asks. You build the folder on a computer, move it to the iPhone, and import it once.

What you are building

Every vault has the same shape. The names in `manifest.json` have to match the files exactly, but the file names themselves are yours to choose.

```
rtu-service-vault/  
├─ manifest.json           names everything below  
├─ safety.md              core: always in context  
├─ error_codes.md         core: fault codes by model  
├─ models.md              core: nameplate quick reference  
├─ documents/             reference: retrieved per question  
│   └─ RTU-500-service-manual.pdf  
│   └─ RTU-500-wiring.pdf  
│   └─ RTU-700-install-guide.pdf  
└─ procedures/            optional: step-by-step flows  
    └─ low_pressure_diagnostic.json
```

The two kinds of content behave differently, and getting the split right is most of the work.

CORE FILES

Short, always loaded

- Read into the assistant's context on every single question.
- Keep the total under about 32 KB. The importer warns above that.
- Right for safety rules, fault-code tables, model and refrigerant quick references.
- Technicians can edit these in the app; edits are kept separately from your originals.

DOCUMENTS

Long, retrieved on demand

- Chunked and indexed on the phone at import. Only the passages relevant to a question are used.
- Every answer drawn from them carries the manual's title and page number.
- Right for full service manuals, wiring guides, installation and commissioning documents.
- No practical size limit; a 300-page manual is fine.

Before you start

1. Install the TestFlight build on the iPhone and pair the glasses.
2. Open **Settings** › **Field Assist**, paste the licence code you were sent, and tap **Activate Licence**. The status card should show **Tier: Team**. Custom vaults and manual indexing need a team licence; a solo purchase covers only the bundled vaults.
3. Turn on **Enable Field Assist** on the same screen.
4. Have a way to get a folder onto the phone: iCloud Drive in the Files app is the simplest, AirDrop of the whole folder also works.

Step 1 · Prepare the manuals

The importer reads the text layer of a PDF. Open each manual on your computer and try to select a sentence with the cursor. If the text highlights, the page

imports as-is. If nothing selects, the page is a scanned image. Scans still import; you have three ways to handle them, and you can mix them per manual.

Route A · Let the phone read it

Nothing to install. Import the scanned PDF like any other. The app reads scanned pages itself during import, page by page, on the device. A 300-page scan takes minutes; keep the app open, and if it is interrupted the import resumes where it stopped. The vault row afterwards shows how many pages were read this way and how many came out low confidence, so you know which manual to replace with a better copy. Any answer drawn from a recognised page carries a note to verify figures against the printed page, because numbers are where recognition fails quietly.

Route B · Add a text layer on any computer (recommended for anything you will reuse)

Run the scan through an OCR tool that writes a searchable PDF, then import that PDF. The pages stay the same, so citations still match the printed book, and the phone does no recognition at all. Free, works on Windows, Mac and Linux:

```
ocrmypdf --language eng RTU-500-service-manual-scan.pdf RTU-500-service-manual.pdf
```

OCRmyPDF is at ocrmypdf.readthedocs.io; on Windows install it through the instructions there (it runs under Python, with Tesseract as the recognition engine). Adobe Acrobat's "Recognize Text" does the same job if you already have it, and so do most scanner drivers' "searchable PDF" settings if you are scanning the book yourself. Whatever tool you use, open the result and select a sentence to confirm the text layer is there before you copy it into `documents/`.

Route C · The repository's extractor, on a Mac

If you have a Mac, the app's repository includes a script that does the same per-page read and writes Markdown with a `Page N` line before each page, which lets you open the text and correct a misread figure before anyone relies on it. It is not in the app; download it from the public repository:

```
curl -O https://raw.githubusercontent.com/straff2002/OpenGlasses/main/Scripts/extract-manual-text.swift
swift extract-manual-text.swift RTU-500-service-manual.pdf
```

It needs Apple's command-line developer tools (`xcode-select --install`). It writes `RTU-500-service-manual.md` beside the PDF and prints how many pages were recognised and how many came out low confidence. Put the `.md` in `documents/` instead of the PDF and give it the same title.

Page numbers are preserved automatically either way, so a citation reads *RTU-500 Service Manual, page 42* and the technician can open the paper copy to it. That only works if the PDF's pages match the printed pages, which is true of nearly every OEM PDF.

A few habits that pay off:

- **One file per manual, one manual per model family.** A combined 900-page binder still works, but a technician hears the title in every citation, so shorter, specific titles read better.
- **Trim what does not help in the field.** Warranty terms, ordering forms and marketing pages add index time and add nothing to answers. Delete those pages before you copy the file in.
- **Keep fault-code tables in the core too.** The manual's fault-code chapter is retrieved fine, but a code that also sits in `error_codes.md` is found instantly by the nameplate lookup without a search at all.

Step 2 · Write the core files

Markdown, plain text, any editor. Two conventions matter because the app relies on them.

Headings define lookup sections

When a technician says "*look up fault T02*" or points the glasses at a display, the app splits each core file at `##` and `###` headings and returns the whole section that contains an exact match for the code. So group codes under a heading per

manufacturer or model, and put the code in a table row or at the start of a line. A section that runs on for pages will be read back in full, so keep sections tight.

```
# Fault Codes
```

```
## Acme RTU-500 (Series 5 controller)
```

Code	Meaning	First check	
ZX9	Low refrigerant charge	Sight glass, then subcooling before adding	
ZX3	Condenser fan failure	Fan capacitor, fan relay	
ZX1	Discharge line sensor fault	Sensor resistance vs. table on page 61	

```
## Acme RTU-700 (Series 7 controller)
```

Code	Meaning	First check	
F12	High pressure lockout	Condenser coil, fan staging	

A safety file the assistant repeats before risky steps

The assistant is instructed by the manifest to remind the technician of safety steps before opening electrical panels or the refrigerant circuit. Put the reminders you want spoken in `safety.md`, in your own words and to your own site rules. Lockout, PPE, recovery before opening the circuit, gas isolation for furnaces, and anything specific to Manitoba code belong here.

A nameplate reference

`models.md` is where model and serial patterns live: what the numbers on the plate mean, refrigerant per model, nominal charge, controller type. When the camera reads a plate, the model number is matched against these sections the same way a fault code is.

Step 3 · Write the manifest

This is the whole file for the folder above. Every key that matters is commented.

```

{
  "id": "acme_rtu",                                lowercase, no spaces; never change it after
import
  "name": "Acme RTU Service",                      what technicians hear: "start a session in Acme
RTU Service"
  "version": "1.0.0",                              bump when you re-import
  "files": ["safety.md", "error_codes.md", "models.md"], core, in the order you want them
read
  "documents_dir": "documents",
  "documents": [
    { "file": "RTU-500-service-manual.pdf", "title": "RTU-500 Service Manual", "kind":
"service_manual" },
    { "file": "RTU-500-wiring.pdf",          "title": "RTU-500 Wiring",          "kind":
"wiring" },
    { "file": "RTU-700-install-guide.pdf",   "title": "RTU-700 Install Guide",   "kind":
"install_guide" }
  ],
  "procedures_dir": "procedures",                omit if you have none
  "gating": { "iap": "enterprise" },             always this for a custom vault
  "prompt_rules": [
    "Never fabricate equipment data, fault codes, refrigerant properties, or procedures.",
    "Use only the vault contents, the retrieved manual passages, and the technician's stated
observations.",
    "Cite the source file or manual page on every factual claim.",
    "If the manuals do not cover the question, say so and recommend escalation rather than
guessing.",
    "Remind the technician of the relevant safety steps before any action that opens an
electrical panel, the gas train, or the refrigerant circuit."
  ],
  "source_attribution_format": "Source: {files}",
  "source_attribution_required": true
}

```

The importer refuses a manifest whose rules do not mention fabrication and citation. Your `prompt_rules` must include a rule containing the word *fabricate* and one containing *cite*. The rules above satisfy that; reword them freely as long as those two words survive.

The title is the citation. Whatever you put in `title` is what the technician hears: *"Source: RTU-500 Service Manual, page 42"*. Name it the way your people already refer to the book.

Step 4 · Procedures (optional)

A procedure is a branching checklist the assistant walks a technician through by voice: *"next"*, *"go back"*, *"repeat that"*, and branch choices from what the technician reports. Skip this for your first import; add one once the question-and-answer loop works.

Each procedure is one JSON file in `procedures/`. The importer checks the graph before install: the entry step must exist, every branch must point at a real step, every non-terminal step needs a way forward, and at least one terminal step must be reachable.

```

{
  "id": "rtu500_low_charge",
  "title": "RTU-500 Low Charge Diagnostic",
  "version": "1.0.0",
  "description": "Confirm and correct a ZX9 low-charge fault on the RTU-500.",
  "safety_notes": [
    "Lock out power before opening the control panel.",
    "Recover refrigerant before opening the circuit."
  ],
  "entry_step": "confirm_fault",
  "steps": [
    {
      "id": "confirm_fault",
      "title": "Confirm the fault",
      "instruction": "Read the controller display. Is ZX9 active now, or only in the history log?",
      "expected_input": "active or history",
      "citations": ["error_codes.md"],
      "branches": [
        { "id": "active", "condition": "ZX9 is active now", "next": "check_sight_glass" },
        { "id": "history", "condition": "ZX9 is only in the log", "next": "intermittent" }
      ],
      "default_next": "check_sight_glass"
    },
    {
      "id": "check_sight_glass",
      "title": "Check the sight glass",
      "instruction": "With the unit running for ten minutes, look at the liquid line sight glass. Is it clear, or bubbling?",
      "expected_input": "clear or bubbling",
      "safety_note": "Keep hands clear of the condenser fan while the panel is open.",
      "citations": ["RTU-500 Service Manual"],
      "branches": [
        { "id": "bubbling", "condition": "Bubbling or foaming", "next": "measure_subcooling" },
        { "id": "clear", "condition": "Clear", "next": "sensor_check" }
      ],
      "default_next": "measure_subcooling"
    },
    {
      "id": "measure_subcooling",
      "title": "Measure subcooling",
      "instruction": "Read liquid line pressure and temperature and tell me both. The manual's target is on page 58.",
      "expected_input": "liquid pressure and temperature",
      "citations": ["RTU-500 Service Manual"],
      "default_next": "done_charge"
    },
    { "id": "sensor_check", "title": "Sensor check", "instruction": "The charge looks right. Check the low-pressure transducer per page 61 before adding refrigerant.", "terminal": true, "outcome": "sensor_suspected" },
    { "id": "intermittent", "title": "Intermittent fault", "instruction": "Clear the log, run the unit for a full cycle, and re-run this check if ZX9 returns.", "terminal": true,

```



```
"outcome": "deferred" },
  { "id": "done_charge", "title": "Charge corrected", "instruction": "Adjust charge to the
target subcooling, record before and after readings, and clear the fault.", "terminal": true,
"outcome": "resolved" }
]
}
```

Step 5 · Import on the iPhone

1. Copy the whole folder into iCloud Drive (or AirDrop it to the phone and save it in Files). Do not zip it; the importer takes a folder.
2. Open **Settings** › **Custom Vaults** and tap **Import Vault Folder...**, then pick the folder.
3. The app validates first. If anything is wrong it names the file and the problem, and nothing is installed. Fix it on the computer and try again.
4. On success the vault appears in the list and indexing starts immediately. Each manual shows a progress bar and then a section count, for example *RTU-500 Service Manual · 412 sections*. Leave the app in the foreground until every manual has a count.
5. Warnings are advisory. The one you are most likely to see says the core files exceed the budget; move long material into `documents` and re-import.

Step 6 · Test the loop

1. In **Settings** › **Field Assist**, choose your vault as the default.
2. Say *"Start a session in Acme RTU Service."*
3. Ask about a code you know is in the manual: *"What does fault ZX9 mean on the RTU-500?"* The answer should end with a source line naming the manual and page.
4. Look at a nameplate and say *"Look up this nameplate."* The app reads the plate on the phone, matches the model against your core files, then against the manuals.
5. Ask something the manuals do not cover: *"What is the torque spec for the compressor terminal cover?"* if that is not in the book. The correct answer is that the loaded manuals do

not cover it and to escalate. If you get a confident answer instead, tell us; that is the behaviour the vault rules exist to prevent.

6. End the session. The audit log records every question, answer, photo and citation, and a team licence can export it as PDF.

Everything stays on the phone. Manuals are indexed and searched on the device and never uploaded to us. The passages relevant to a question are sent to whichever AI model you configured, with the question, in the same way the rest of the app works. If you use an on-device model, nothing leaves the phone at all.

Updating a vault

Edit the folder on the computer, bump `version`, and import again with the same `id`. Manuals whose content has not changed are skipped; a changed manual is re-indexed and its old index removed; a manual you dropped from the manifest is removed. Technicians' in-app edits to the core files are kept and layered over your new originals. To pull a vault off the phone with those edits included, swipe the vault row and choose **Export**; the exported folder imports straight back in.

If the import refuses

MESSAGE

manifest.json is missing or unreadable

listed file missing: models.md

listed document missing: documents/...

... has no text layer on N of N pages; it will be read by on-device recognition at impo

prompt_rules should address 'fabricate'

document has no title

... → unknown step

no terminal step is reachable

core files total ... characters (budget 32768)

Turning a vault into a pack

Once a vault has proved itself with your own crews, the parts you authored, the core files and the procedures, can be published as a **pack** that other Field Assist users install from the app's Packs list, and that a team licence can include for a whole organisation. A pack never contains manufacturer manuals; a customer who installs your pack adds their own manuals to it exactly as described above. What you are publishing is your fault-code tables, nameplate decoding, safety rules and diagnostics, which is where your field knowledge lives.

1. Copy the vault folder and delete `documents/`. Remove `documents` and `documents_dir` from the manifest, and set `gating` to `{ "iap": "com.openglasses.vault.<id>" }`, where `<id>` is the vault's `id`.
2. Add a `pack.json` beside the manifest.

```
{
  "id": "com.openglasses.vault.acme_rtu",
  "vaultId": "acme_rtu",
  "version": "1.0.0",
  "name": "Acme RTU Service",
  "summary": "Fault codes, nameplate decoding, and diagnostics for Acme rooftop units.",
  "author": "Your company name, as technicians should see it",
  "licensePack": "acme_rtu"
}
```

3. Send the folder to us. We sign it, host it, and list it in the catalog; nothing unsigned can be installed. If the pack includes extracted manual text, it needs the manufacturer's written permission to redistribute, recorded in the pack, and we will ask for it.

4. Decide how it is sold. A pack is its own App Store product for solo technicians, and a team licence includes it when the code is issued with the pack's key, so your own customers get it with their licence rather than buying it separately.

Updating a published pack is the same folder with a higher `version` .

Technicians who install the update keep their in-app edits and their own manuals.

Checklist before you send a vault to a customer

☐ Every PDF selects text on the computer, or you have decided to let the phone read it.

☐ Each manual's `title` is the name the crew already uses.

☐ Fault codes sit under a heading per model, one code per row.

☐ `safety.md` says what you want spoken, in your words.

☐ Core files total well under 32 KB.

☐ Import shows a section count for every manual.

☐ A known fault code answers with a page citation.

☐ An out-of-scope question answers with "the manuals do not cover this".
